Adaptive Cruise Controller for RC Car Using STM32

Erickson A. Mijangos
Department of Electrical Engineering
California State University Fullerton, Fullerton, California
eamijangos@csu.fullerton.edu

Dr. Mike Turi
Department of Electrical Engineering
California State University Fullerton, Fullerton, California
mturi@fullerton.edu

***Abstract*—This project documents the design and development of an RC Car built from the ground up, as an attempt to implement and test an Adaptive Cruise Control System. Two STM32 Nucleo-Boards from STMicroelectronics are used: the STM32L432KC for the transmitter and the STM32H723ZG for the RC Car. A Python User Interface is developed for controlling the RC Car and performing data acquisition in order to analyze and develop the adaptive cruise controller.**

The sensors integrated to the RC Car are the LIS3DH accelerometer and the Garmin LIDAR Lite V3. The accelerometer was used to determine the velocity of the RC Car, and the LIDAR to measure the relative distance to a leading vehicle ahead. During testing, two challenges arose: the velocity calculated by the accelerometer drifted from the true value over time due to sensor noise, and the NRF24L01 lost communication for 1–1.5s every 20ms cycle which affected velocity calculation.

Due to Earth's gravitational influence on the accelerometer, a calibration feature was implemented, but the communication losses made it difficult to confirm its effectiveness. Based on the test results, the adaptive cruise controller could not be fully implemented. Future work includes integrating a Hall Effect sensor and applying a Kalman Filter for sensor fusion to improve velocity accuracy and enable closed-loop speed control.

***Index Terms*—Adaptive Cruise Control, STM32, PID Controller, LIDAR, NRF24L01, PyQt5, UART.**

I. Introduction

The purpose of this project is to explore electronics, software development, sensor integration, numerical analysis, real-time data acquisition, and primarily control engineering by building an RC Car from the ground up. The RC Car is designed to develop an Adaptive Cruise Controller with data acquisition in order to properly implement it. Figure 1 shows a high-level overview of the entire project — the RC Car listens and sends data back to the transmitter, which is connected to a Python User Interface. The RC Car and transmitter communicate using the NRF24L01 programmable RF module, and the transmitter connects to the computer via USB using the UART protocol to send commands and receive data.

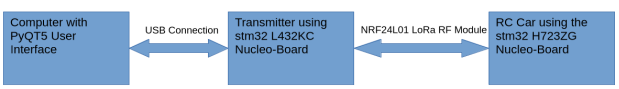

Fig. 1. High-level overview of the project architecture.

The setup of the Transmitter and Python User Interface is shown in Figure 3, and the RC Car with the transmitter is shown in Figure 4. The RC Car integrates the LIS3DH accelerometer to measure velocity and the Garmin LIDAR Lite V3 to measure relative distance.

Based on a simulation study conducted in MATLAB/Simulink at CSULB[1], the Adaptive Cruise Control system is designed to autonomously maintain the desired velocity while detecting and measuring the relative distance to a leading vehicle ahead. As shown in Figure 2, the relative distance and vehicle speed are analyzed to tune the adaptive cruise controller.

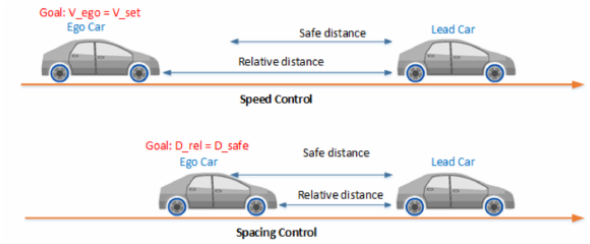

Fig. 2. Speed/Spacing Control to setup Adaptive Cruise Control.

To implement the adaptive cruise controller in hardware, the Garmin LIDAR Lite V3 serves as the primary distance sensor, measuring the relative distance between the Ego Car (the RC Car under ACC control) and the Leading Car (a second RC Car given a specified motion). The LIS3DH accelerometer measures the Ego Car velocity, in which a PID controller can be developed to make the RC Car autonomous. The Leading Car will be given a speed with a small oscillation to simulate realistic driver behavior, as shown in Figure 5.

To achieve the simulated behavior of the adaptive cruise controller, the control system shown in Figure 6 models the Leading Vehicle motion and the Sensor Model, which depends on the relative position and velocity of the leading vehicle and the ego vehicle, so it can be fed into the ego vehicle plant to predetermine next motion. The input signals measured from the relative and ego vehicle parameters are used to determine the ego vehicle Adaptive Cruise Controller are shown in Figure 7.

[1]Full report available in the project repository: `EE471_ACC_CSULB.pdf`.

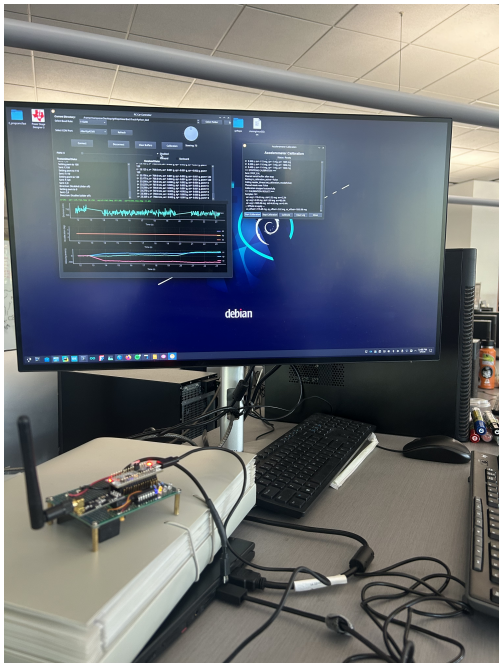

Fig. 3. The Transmitter connected to the Python user interface.

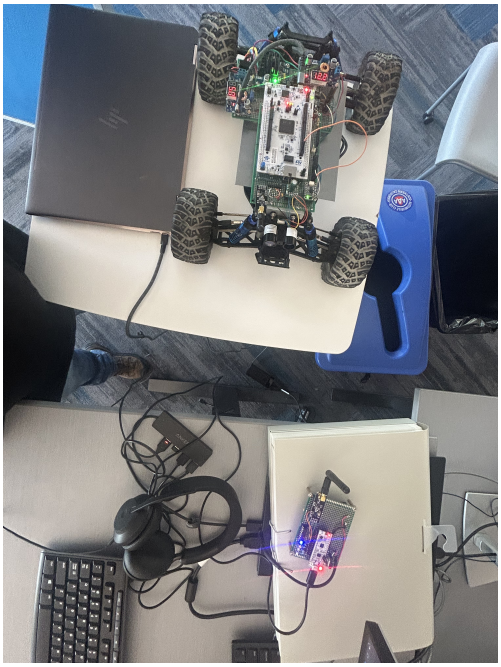

Fig. 4. The Transmitter and RC Car.

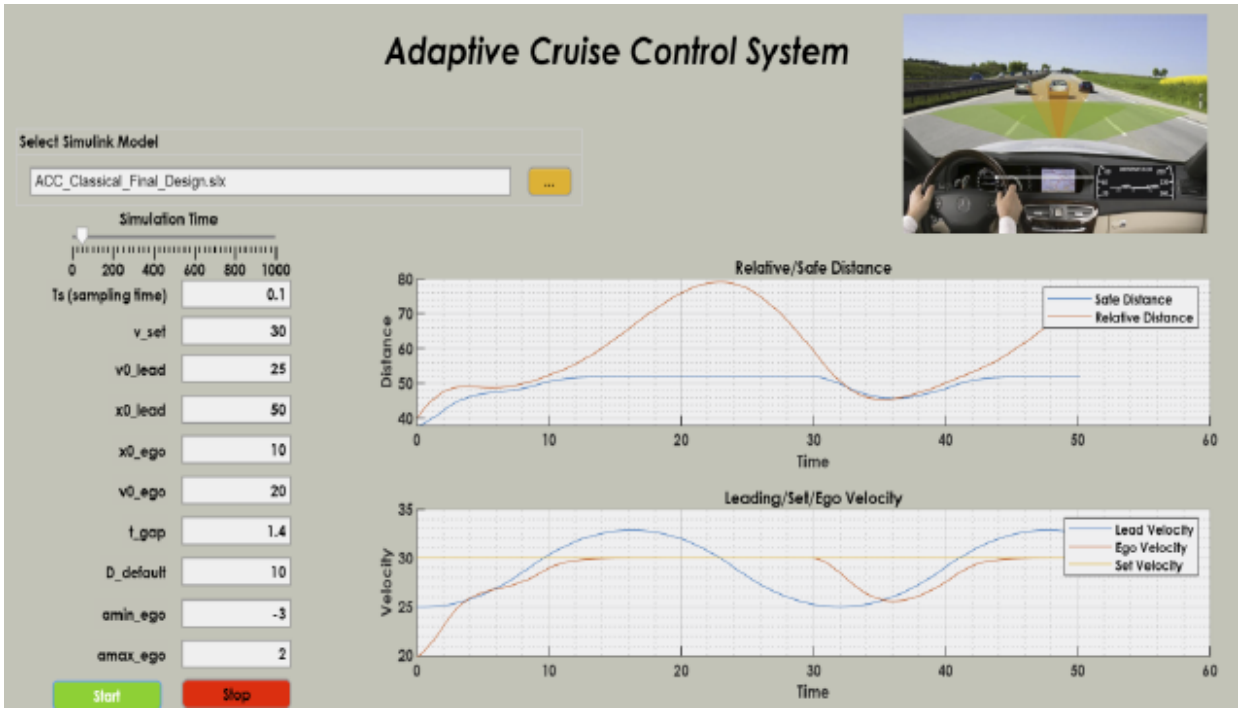

Fig. 5. The User Interface developed in MATLAB and Simulink.

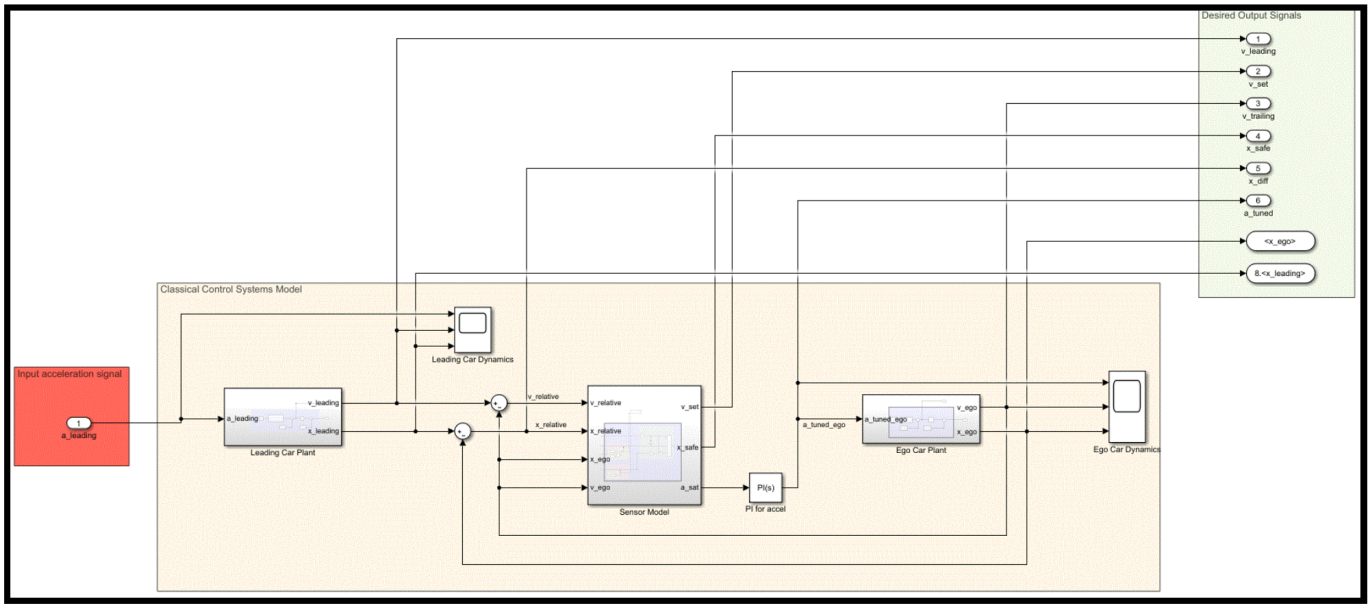

Fig. 6. High-level overview of ACC model.

To place the adpative cruise controller into test, data acquisition with a descent sample rate from both accelerometer and LIDAR are required. To analyze and process the signals collected, the python user interface needs to be implemented in order acquire data in real time so a proper adaptive cruise controller can be implemented.

II. Hardware Architecture

A. RC Car Design

The RC Car uses the STM32H723ZG Nucleo-Board, due to having the 550MHz clock speed, to achieve better timer control if necessary, minimize the execution time per instruction/code, accessibility of more peripherals, and the larger system RAM to store large amount of data for testing purposes. The peripherals used in this MCU are SPI, USART, I2C, timers, and Direct Memory Access to either control sensor and transmit data during development. In Figure 9, shows the STM32H723ZG Nucleo-Board integrated with all the sensors, and power system section along with the RC Car with the all sensors/modules labeled in Figure 8.

The Entire RC Car is powered with a 12V battery, but the RC Car is divided by subsections based on the schematic. On the left there a yellow box is drawn to show the MCU along with its sensors, and on the right in the green box shows the power system and the motor driver. The power section is electrically isolated due to the FOD8001 Logic Optocouplers, and the TMH1205S 2W Traco Isolated DC-DC Converter that isolate the power ground and digital ground on the MCU section. This feature is added due to capability to troubleshoot, isolate noise that can influence the sensors during data transmission, and if any power section components/modules are destroyed it would not destroy the MCU and the sensors.

As labeled in the RC Car schematic the sensors used are labeled to the their corresponding pins on the MCU and their protocols. The NRF24L01 RF module uses the SPI protocol where the data rate is set to 3.9 Mbit/s. According to the datasheet of the NRF24L01 would need 130us to switch RX or TX mode, and also to including the execution timing of the program which can add more time. When bidirectional communication was first tested, the bare minimum to have

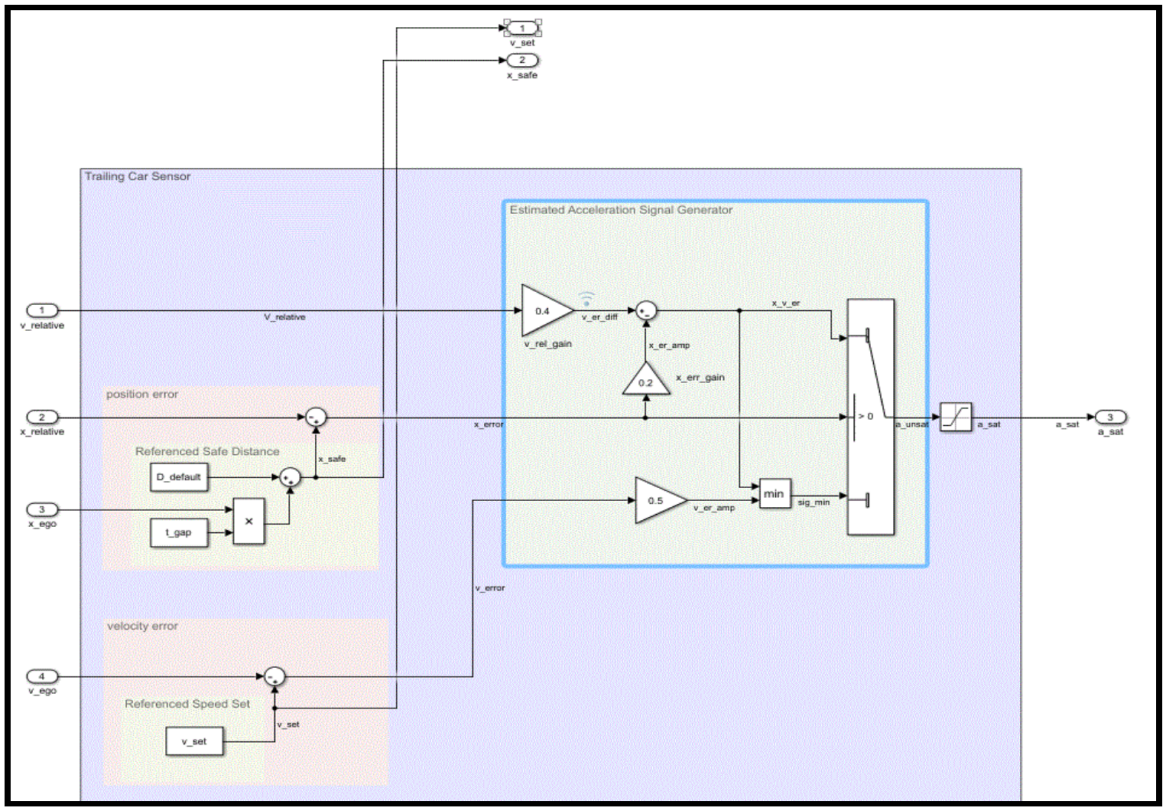

Fig. 7. Block diagram of vehicle plant and derivation for velocity and position motion.

a reliable communication is 4ms. To provide reliable bidirectional communication for most of the time is 10ms before toggling RX/TX mode. The 10ms toggling time also considers the amount of time to read the sensor data and package the data for transmission. On the MCU, three built-in LEDs are used to track the state of the RC Car firmware. Adding the status LEDs helps the user keep track and troubleshoot the RC Car during the development or issue.

- **Yellow LED** — Transmitter is in TX mode (sending sensor data to transmitter)
- **Red LED** — Data packet transmitted from RC Car
- **Green LED** — Command received from transmitter for RC Car next action

The remaining two sensors that uses I2C protocol are the LIS3DH accelerometer, and Garmin LIDAR V3 Lite. The clock speed that the MCU can configure for I2C protocol is either 100kHz or 400kHz. Both sensors are clocked to 400kHz, and was confirmed that it is reliable when both sensors library were written. The perf board that holds the MCU, have the wires soldered and close to the MCU in order to ensure I2C is reliable and minimize the stray capacitance that can affect the rise and fall times of both data and clock lines. Both sensors would be sampled when it is in the state to transmit data to the transmitter which is every 20ms.

With the MCU and the sensors isolated from the power section, the FOD8001 optocouplers and servo motor needs to still be powered. The 12V serving as the main power source of the RC Car, the MCU, optocouplers, and motor driver still need to be powered properly by adding voltage regulators, and DC/DC converters. The optocoupler on the power section, still needs to be powered about 5V so that the PWM pulse can be passed to the motor driver and servo motor.

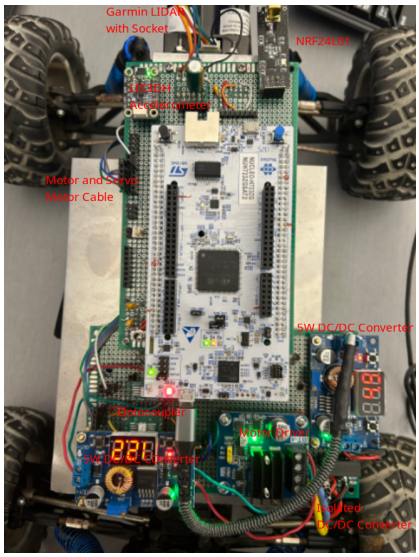

Fig. 8. RC Car Labeled.

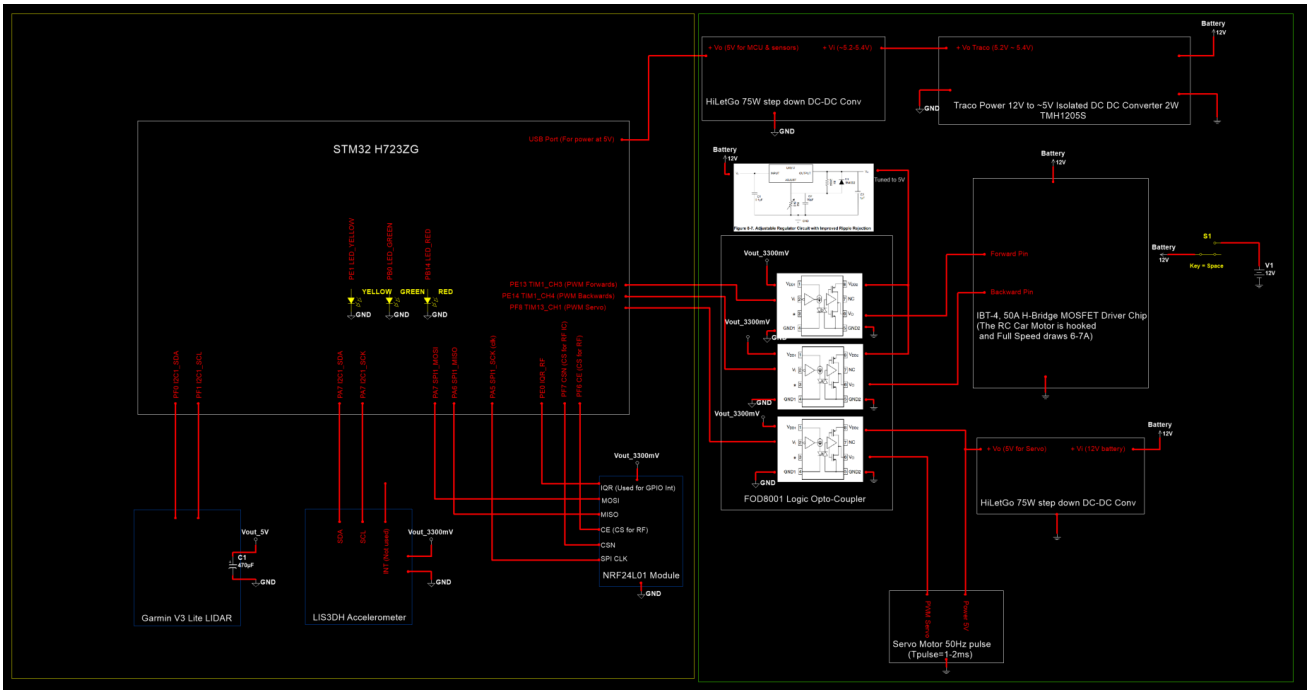

Fig. 9. The Schematic of RC Car.

Two of the optocouplers responsible to pass the PWM pulses to the MOSFET motor driver, are powered by the LM317 voltage regulator, where it can be tuned 5V based on the adjustable circuit from the Texas Instruments datasheet as shown in the schematic. For both the third optocoupler and servo motor, are powered by the 75W HiLetgo DC/DC stepdown converter. Although the both the third optocoupler and servo motor can be powered the LM317, it was not due to potential concerns that the it would heat up and go to thermal runaway. The servo draws about 500-700mA when it was manually tested with 5V supply along with function generator, generating the PWM that is pulsing within 1-2ms, with a time period of 20ms (PWM frequency is 50Hz). Since the LM317 drops the 12V to 5V, it would have a 7V drop across it and if more current is drawn it would dissipate more heat. As a result it is a safer route to use the 75W HiLetgo DC/DC stepdown converter to power the optocoupler along with the servo motor.

The 75W HiLetgo DC/DC stepdown converter can display the input and output voltage it was provided. The battery can be monitored from checking the HiLetgo DC/DC converter that is intacted to the servo motor and third optocoupler but toggling the output push button. The traco isolated DC/DC converter is stepdown to about 5.2V-5.4V from the 12V battery, but a HiLetgo DC/DC converter is added between the output of the isolated DC/DC converter and the MCU so that it can be tuned to 5V. It can be powered by the isolated DC/DC converter but according to the schematic of the H723ZG Nucleo-Board on the Micro-USB port it has a zener diode to clamp the supply voltage of the entire board to 5V and would be safer to tune it at 5V. When the battery runs low, the bare miminum voltage that be supplied to the MCU to work proper is 4.6V before it powers off and become unreliable.

B. Transmitter Design

Compare to the RC Car, the schematic of the transmitter is simpler as shown in Figure 11, along with the actual labeled transmiter in Figure 10. The Nucleo-L432KC board was chosen for the transmitter due to it small Ardunio Nano like size, and the max clock speed is 80MHz. The performance does not need to be robust compared to Nucleo-H723ZG board since primarily it would be toggling every 10ms from RX/TX mode and USART interrupts are used to send commands to transmitter from the Python User interface and transmit data it has receive to the User Interface. The transmitter is connected to the computer to transmit/receive data by using the UART protocol with a baud rate of 115200 bits/s.

Fig. 10. Transmitter Labeled.

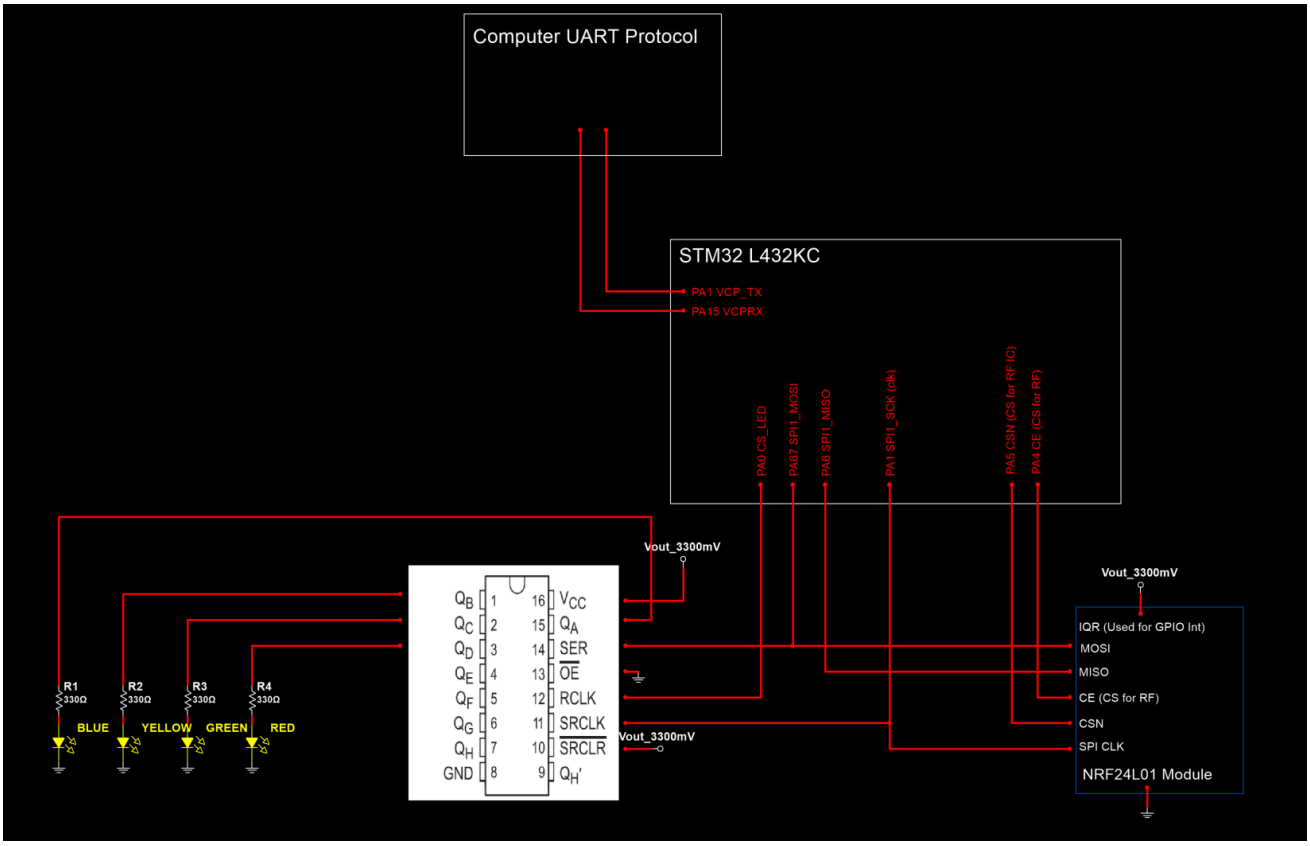

Fig. 11. The Schematic of Transmitter.

Along with the NRF24L01 RF module, a shift register is added to control four status LEDs where:

- **Blue LED** — Transmitter is in RX mode (listening for RC Car data)
- **Yellow LED** — Transmitter is in TX mode (sending command to RC Car)
- **Red LED** — Data packet received from RC Car
- **Green LED** — Command received from Python GUI and ready to transmit

Adding the status LEDs help the user track the actions of the transmitter for trouble-shooting and development.

III. Firmware Design

A. RC Car Firmware Design

Based on the hardware integration of the RC Car, the firmware designed in Finite State Machine (mostly behave as Mealy State Machine where next state varies on both current states and inputs to determine the state). The heart of the firmware is Timer 3, where it will toggle between RX/TX mode and will determine the next action based on the current inputs it has receive during any current state it is in. Figure 12 shows the RC Car Finute State Machine logic where in the center most of the action occurs the Timer 3 periodic interrupt to toggle communication. To simplify the logic further pseudocode is added to conditionally determine the RC Car next action.

The movement of the RC Car including the forward, backward, coast, brake,and steer states will vary on the next state variable which is determine what command it has received in the Receive state. If the RC Car is in disable mode, it will keep the RC Car in coast mode next state variable will be marked as coast state and it will not transmit data back to the transmitter. In the Transmit state there is a stopStateFlag where that will keep the RC Car disabled, if next state is marked as wait state, then it will signal the RC Car stay stationary and not transmit sensor data.

When the RC Car receives either forward, backward, or steering commands it will start to transmit sensor data. In the receive state for the entire 10ms, it will be in a poll state until it has receive a command from the NRF24L01 RF module. If a command is receive, it will mark the current state to get sensor data and the next state after it has acquired sensor data to the RC Car Action which either can forward, backwards, or steering.

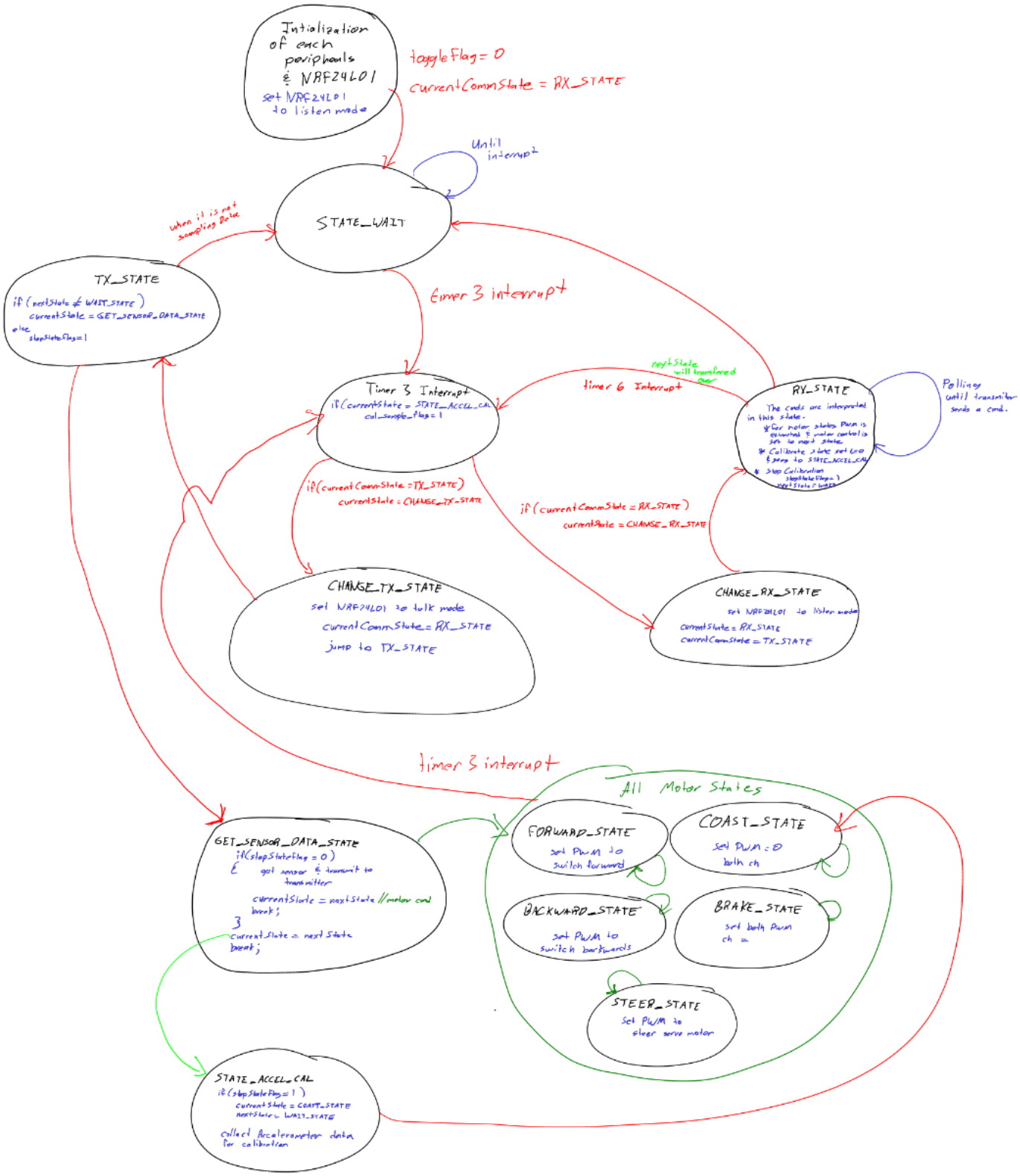

Fig. 12. The finite state machine of the RC Car.

Fig. 13. The finite state machine of the Transmitter.

In the get sensor data state, it checks if stopStateFlag where if set to zero, it can start getting the data from the sensors. It was added due to the calibration feature that was added, for calibrating the accelerometer. The difference is that the get state will package the data in a 32-byte format in the following notation:

```
timestamp_ms,distance_cm,ax_mg,ay_mg,az_mg\n
```

Whereas the calibration format is excluding the LIDAR measurements in the following notation:

```
timestamp_ms,ax_mg,ay_mg,az_mg\n
```

The calibration state was added since the accelerometer is measuring Earth gravity, and since change of motion is desire the accelerometer is transmitted to calculate the average offset.

B. Transmitter Firmware Design

The transmitter finite operates in similar fashion (Figure 13) with the difference off only listening for data, and transmitts data when a USART interrupt occurs with a baud rate of 115200 bits/s. The USART interrupt, signals the MCU that it has received a command from the Python User interface will prepare to transmit command when it switches to the transmit state. When data gets received during the receive state, it will transmit the data it has receive to the Python User interface.

IV. Python Graphical User Interface

The transmitter will transmit commands to the RC Car, and receive data from the RC Car under the Python User interface. Figure 14. The package used for the user interface is the PyQt5, PyQtgraphs for the real time plots, and PySerial to perform UART communication between the user interface and the transmitter. The action that were performed are displayed in two textboxes labeled Transmitted Box Data which shows which buttons are pushed and sent to the transmitter and the Receive Data Box to display the data that was receive from the transmitter. Similarly the calibration windows has a textbox to display both actions and data receive based on the buttons pushed.

In order to display the textbox and plot data in real time, the user interface would need to use the QThread which imported under PyQt5 package, otherwise the user interface would not respond. The current flowchart/state diagram does not include the logic of determining whether the data is measurement(all sensors measured) or accelerometer data (for calibration) but the functionality stays the sames as shown in Figure 15. As mention in the firmware section of the RC Car the disable button ensures the RC stays stationary and will collect data when the steering dial is moved, or both forward/backward radio are set with the adjusted slider that will send the PWM value to control speed.

Fig. 14. The Python including Calibration window.

Fig. 15. The state diagram for threading feature when data is received from transmitter.

V. Experimental Results

With the RC Car, transmitter and Python User Interface developed, RC Car was tested to respond on commands it has received and if data is collected and displayed on the User Interface. Figure 17 shows the RC Car on the first test drive based on the video recording for 30s. Based on the video recording there was no obstacle or walls detected by the LIDAR through out the test drive, so the measurement is unreliable and the max distance the LIDAR can read up to 40m. All the axis of the acceleration is acquired, and it can be noted that acceleration measurement received from the accelerometer is measured in mg (1000mg=1g). The orange signal is the acceleration on the z-axis measured which is typical close to 1000mg which indicate Earth gravitation influnce on the accelerometer. Since the csv file was not exported on the time it was tested, it would be difficult to zoom in on the acceleration in the x and y component but the video showed velocity for both x and y component zoomed in Figure 18.

The velocity signal can be calculated numerically using

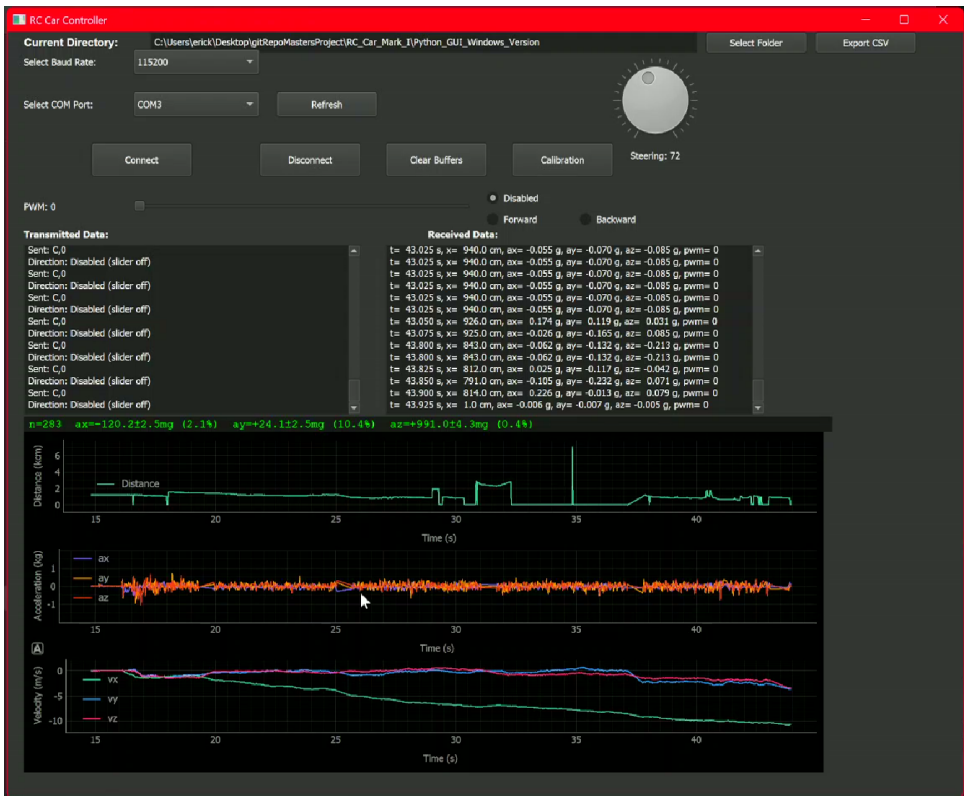

Fig. 16. The second drive test with accelerometer calibrated.

Euler Integration Method as shown below where g is the acceleration due to gravity constant.

$$v_n = v_{n-1} + 0.001g \cdot \cdot a_n \Delta t \tag{1}$$

Since the sample rate of both sensors are 25ms it can be assumed that is good for numerically perform integration. However, the z-axis component since it is consistently about 1g on average, the velocity on the z-axis which is pink is increasing which will indicate that RC Car is falling for 30s, which is not the case in Figure 17. The zoomed plot display the behavior of the accelerometer in the x and y component, where the velocity in the y component is close to 0. This indicate that the RC Car was not tilted for 30s, but the velocity on the x component at the 30s indicate that the car is constantly accelerating and has reached 50 m/s when the test stop which it is not the case. The RC Car is tilted at an angled for the x component where the Earth gravity has influence the accelerometer at the x component. This will affect the velocity on the x component calculation, which is shown on the signal plotted. In this case since the accelerator was theoretically going to calculate the RC Car's velocity, another feature is added so that the gravitional influence can be filtered out. The calibration button is addded so that in a seperate window it can sample the accelerometer data it can calculate the DC bias which are the gravitional components. The average and standard deviation are calculated so DC bias can be filtered out on the accelerometer measurement it has received.

Figure 19 and 20 shows the accelerometer waveforms for about 4s, where the RC Car is stationary. When calibrated, the accelerometer is close 0mg for all x, y, and z, compared to uncalibrated waveform. The average and standard deviation accelerometer display in neon green text above plots, which are the offset values that will be filtered out when accelerometer data is received. In the second video it demonstrates the difference on the velocity calculation compared to the uncalibrated version as shown in Figure 16 shows the second test drive with the offset values filtered from the measurements, where the measured values are reasonable compared to the uncalibrated version that was first tested. ALthough the velocity values are reasonable range, but it suffers with velocity drifting from the true measurements and is increasing over time due to the noise accumulated over time.

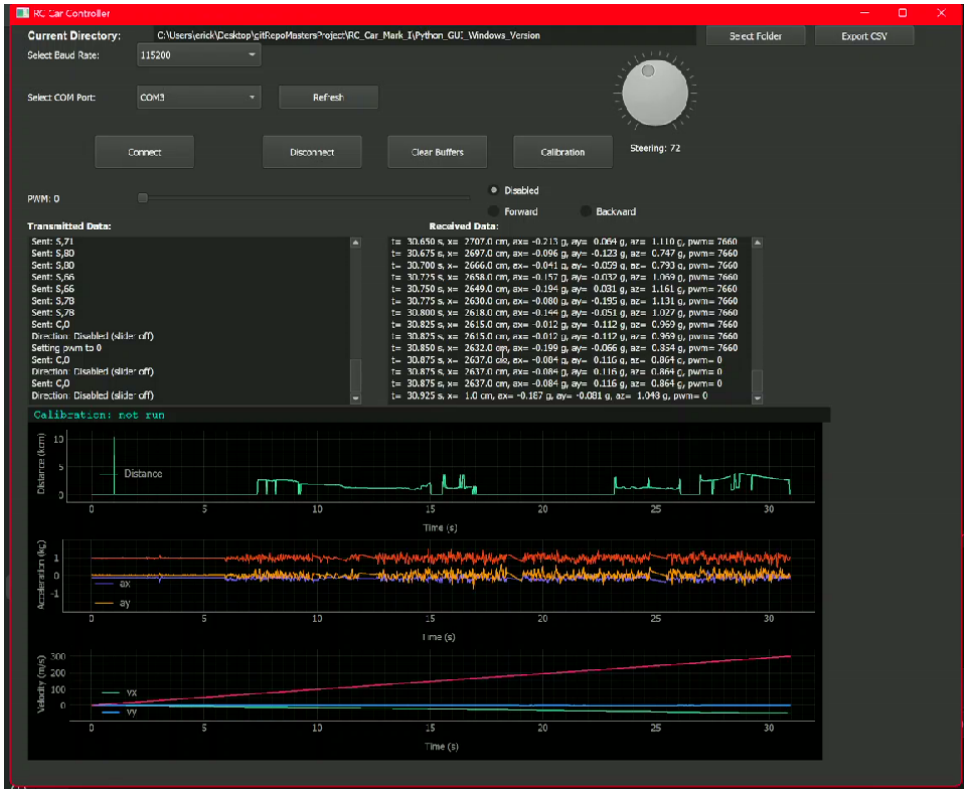

Fig. 17. The User Interface during the 30s test.

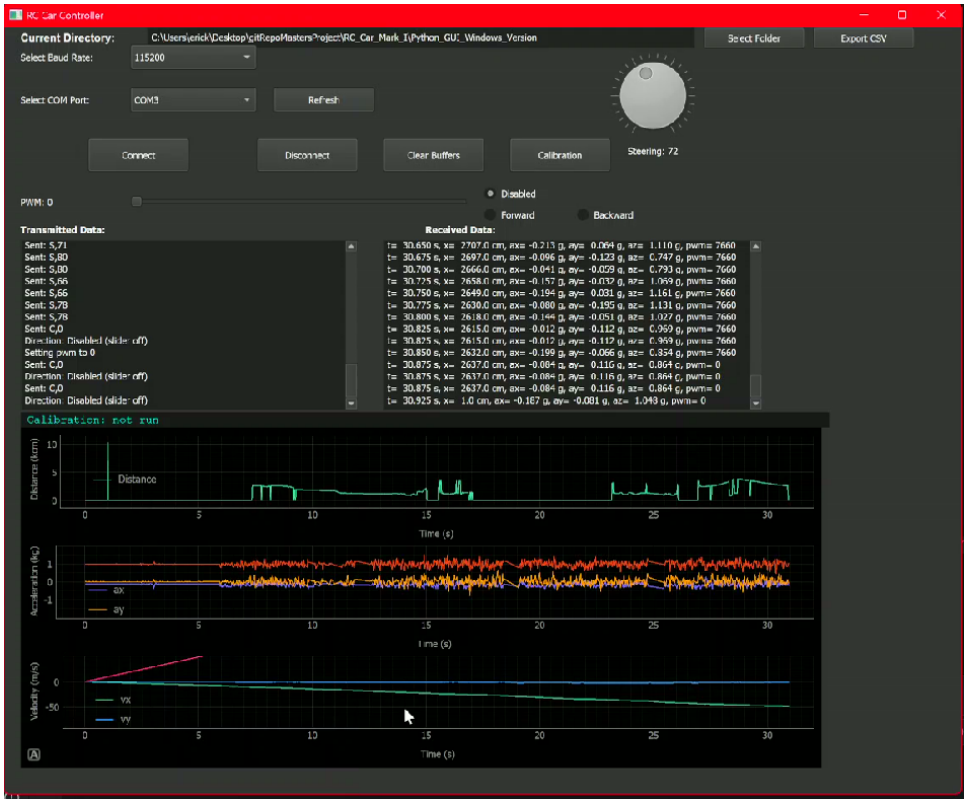

Fig. 18. The same User interface but with the x and y velocity component zoomed in.

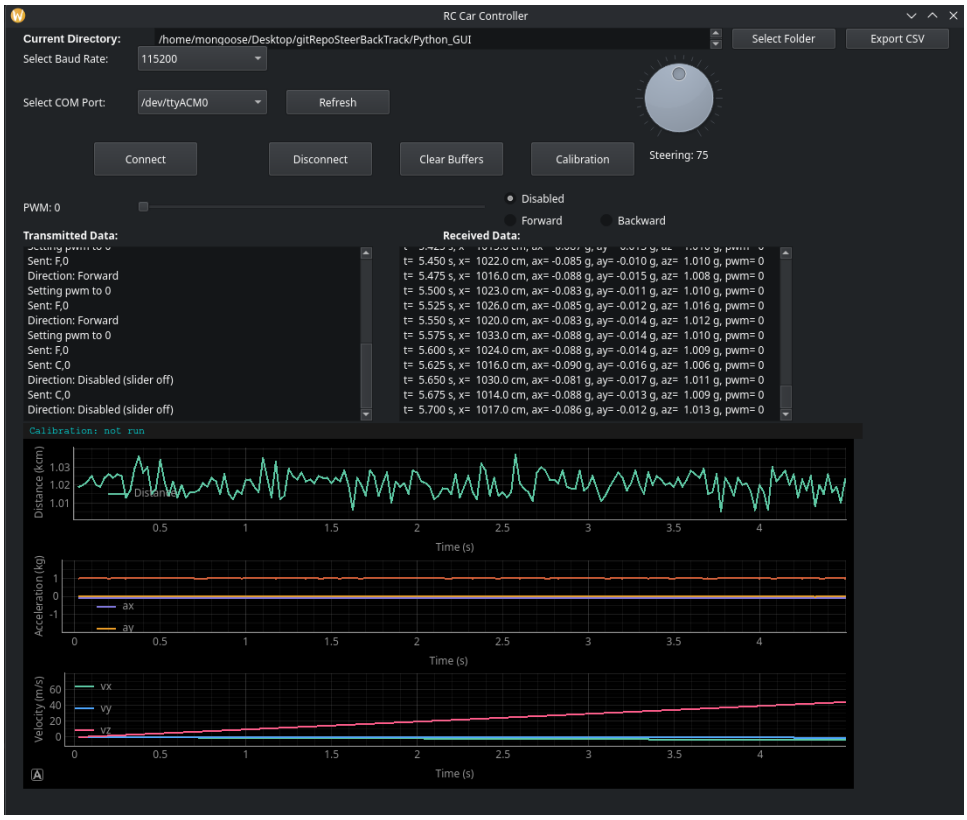

Fig. 19. The same User interface with no calibration.

Fig. 20. The same User interface with calibration.

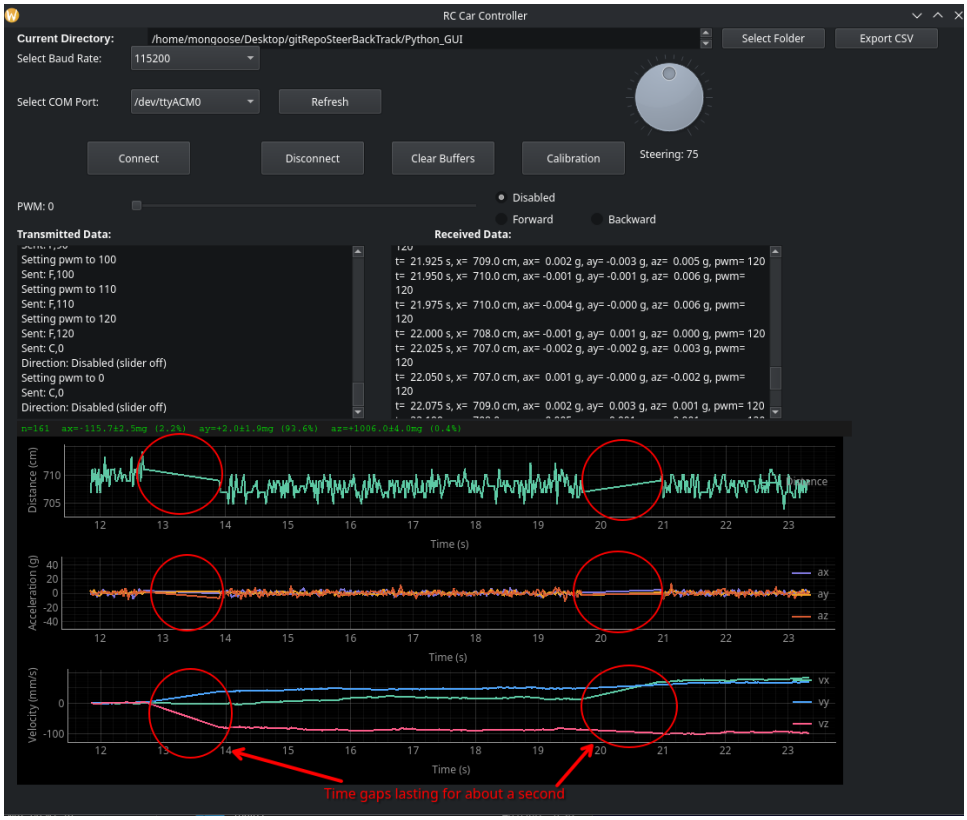

Fig. 21. A sample of waveform where time is occured.

VI. Conclusion and Future Improvements

The RC Car, Transmitter, and User Interface are mostly developed and was able to respond to commands and receive data into the user interface, the adaptive cruise controller could be developed. Two factors affect the development of the adaptive cruise controlle such as data acquisition not sampling consistently, and drifting effect from the velocity calculation. By addressing these two factors, there are potential improvements that can be added to be able to obtain accurate measurements in real time.

Data Acquisition work for most of the time but there time gaps of lost data as shown in Figure 21. This is due to the NRF24L01 from either the RC Car not transmitting, or the transmitter is in talk mode where it could not receive the data. Before integrating the steering mechanism, it use to sample consistently without losing data. During the test the RC crashed during the test and have weaken the socket where the NRF24L01 is placed. In the NRF24L01 library during data transmission of the RC Car to the transmitter, a red built-in status LED of the Nucleo-H723ZG board is signaled that data transmission if the data is package and transmitted otherwise it will block transmission if the red status LED does not toggle. Similar for the transmitter it toggles the status red LED to indicate that data is received. During the time gap the transmitter red LED does not toggle which is a potential indicatation that it could not listen to the data after the RC Car transmit the data. Since it use to consistently collect data, the NRF24L01 for both RC Car and Transmitter would need to be soldered to ensure it has good contact and prevent the issue from occuring. Another would be to probe the status red LEDs on the same logic analyzer to determine if it is the NRF24L01 not configuring either TX/RX mode on time. two of the FOD8001 logic optocoupler can be wired from both RC Car and transmitter so that the both signal share the same ground and would make the troubleshoot easier by comparing the time sequence. With time gap of losing communication, time would drastically affect the velocity calculation since 1-1.5s is lost as shown in the waveform. By fixing the time gap issue, another challenge would be the drifting issue from the integration calculation.

In theory accelerometer has the potential to measure the velocity of the RC Car, but in practice drifting over time is a common issue for IMU sensors. Although calibration fixes most of the velocity by removing the gravitional offsets measured for the accelerometer, it is a band aid solution from the drift effect. The noise plays a huge role on accumulating the drift, the accelerometer is good for detecting jolts (change of rate for acceleration). Integrating numerically the velocity, it is relying on acceleration data but it does not know the initial of the RC Car at the moment of time. For instance, the RC Car can run at a constant speed until it is signal to stop. The accelerometer would detect the deacceleration but would not be enough to state the RC Car has stopped. At zero

acceleration either the RC Car is stationary or moving at a constant speed but it does not know the initial condition at the moment when the integration calculation is performed. One purposal to fix the drifting issue is sensor fusion, where more sensors can be incorporated to describe more parameters of the RC Car.

A magnet can be attached to the wheels and a hall effect sensor can be place near the RC Car to measure the RPM of the RC Car which could eventual observe the velocity of the RC Car. The accelerometer can be replaced with purely with Hall Effect sensors, but at a low RPM the Timer would not be able time the next pulse of the Hall Effect sensor since the timer resets when 2^{16} or 2^{32} ticks has passed. The accelerometer and hall effect sensor can be fused so that it has the initial conditions when calculating velocity and describe the physical state of the RC Car compare to the accelerometer data alone. This is a precursor to Kalman Filters where can be implemented to compare the observed measurement and predicted measurement so that when integration is performed it can correct itself when drift occurs. The mathematics of Kalman Filters can be explore in order to properly implemented so that measurements can become accurate and reliable to develop the adaptive cruise controller.

REFERENCES

[1] STMicroelectronics, *STM32H723ZG Datasheet*, Rev. 4, 2022. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32h723zg.pdf [Accessed: May 2026].

[2] STMicroelectronics, *STM32L432KC Datasheet*, Rev. 7, 2021. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32l432kc.pdf [Accessed: May 2026].

[3] STMicroelectronics, *LIS3DH Datasheet*, Rev. 3, 2016. [Online]. Available: https://www.st.com/resource/en/datasheet/lis3dh.pdf [Accessed: May 2026].

[4] Nordic Semiconductor, *nRF24L01+ Single Chip 2.4GHz Transceiver Product Specification*, Rev. 1.0, 2008. [Online]. Available: https://www.nordicsemi.com/products/nrf24l01 [Accessed: May 2026].

[5] Garmin Ltd., *LIDAR-Lite v3 Operation Manual*, 2017. [Online]. Available: https://static.garmin.com/pumac/LIDAR_Lite_v3_Operation_Manual_and_Technical_Specifications.pdf [Accessed: May 2026].

[6] MathWorks, "Adaptive Cruise Control Using Model Predictive Controller," *MATLAB Documentation*, 2024. [Online]. Available: https://www.mathworks.com/help/mpc/ug/adaptive-cruise-control-using-model-predictive-controller.html [Accessed: May 2026].

CODE AVAILABILITY

The complete firmware, Python GUI source code, schematics, and documentation for this project are publicly available on GitHub: https://github.com/eam95/RC_Car_Mark_I